



Optimax: hoe het in elkaar zit

Uitleg voor niet-technici. DGS productie- en alarmdashboard. Versie 11 juni 2026.

Waarvoor dit document is

Optimax bestaat uit een aantal losse onderdelen die samenwerken. Dit document legt in gewone taal uit welke onderdelen dat zijn, wat elk onderdeel doet, welke kant-en-klare software (een "library") daarvoor gebruikt wordt, en hoe het systeem bijhoudt wat het doet (de logging). Geen voorkennis nodig. Technische termen staan achterin in een woordenlijst.

De rode draad: Optimax is in de kern niet ingewikkeld. Het haalt cijfers uit een database, rekent er wat KPI's mee uit, en tekent daar grafieken van in een webpagina. Daaromheen zit een laag voor veiligheid, een laag voor verpakking (zodat het overal draait), en een laag die bijhoudt of alles goed gaat.

1. De grote plaat

Optimax bestaat uit drie hoofdlagen. Stel je een restaurant voor:

- **De eetzaal (voorkant):** wat de gast ziet en aanraakt. Bij Optimax is dat de webpagina in de browser met de grafieken, tabellen en de chat.
- **De keuken (achterkant):** waar de bestellingen binnenkomen en het eten wordt klaargemaakt. Bij Optimax is dat het programma dat vragen beantwoordt en de cijfers berekent.
- **De voorraadkast (database):** waar alle ingrediënten liggen. Bij Optimax is dat de database met alle productie- en alarmmetingen.

In een tekening:



Belangrijk om te onthouden:

- De browser praat alleen met de keuken (poort 9000). De browser komt nooit rechtstreeks bij de voorraadkast.
- De keuken is de enige die bij de database mag. Dat is bewust: zo kun je niet via de webpagina zomaar bij de ruwe data.
- Alles draait samen op één apparaat bij DGS: de "logger" (een IXON SecureEdge, een soort minicomputer in de fabriek). Je komt er via een beveiligde VPN-tunnel bij.

2. De bouwstenen, stuk voor stuk

De voorkant: wat je in de browser ziet

Bouwsteen	Wat het is	In gewone taal
React	Library om webpagina's te bouwen die meteen reageren	De "decorateur" die de eetzaal inricht en de borden steeds bijwerkt zonder de hele zaal opnieuw te dekken

TypeScript	Programmeertaal voor de voorkant, met ingebouwde foutcontrole	Een tekstverwerker met spellingcontrole: vangt typefouten vóór het de klant bereikt
Vite	Gereedschap dat alle voorkant-code samenperst tot kleine bestanden	De inpakker: maakt van honderden losse blaadjes één compact, snel pakket
Tailwind	Kant-en-klare opmaakregels (kleuren, marges, knoppen)	Een doos lego-stijlblokjes in plaats van elke knop met de hand schilderen
Recharts	Library die grafieken tekent (staaf, lijn, taart)	De tekenaar die van een rij cijfers een grafiek maakt

Wat dit samen doet: je opent de pagina, React tekent de lay-out, Recharts maakt er grafieken van, en Tailwind zorgt dat het er verzorgd uitziet. Vite heeft dat alles vooraf tot een paar kleine bestanden samengeperst zodat het ook over een trage verbinding snel laadt.

De achterkant: de motor

Bouwsteen	Wat het is	In gewone taal
Python	De programmeertaal van de achterkant	De taal waarin de keuken is geschreven
FastAPI	Library die een "loket" (API) bouwt waar de browser vragen stelt	De balie met afgebakende loketjes: elk loket beantwoordt precies één soort vraag
Uvicorn	Het programma dat FastAPI laat draaien en verzoeken aanneemt	De portier die bezoekers binnenlaat en naar het juiste loket stuurt
Pydantic	Controleert of binnenkomende gegevens het juiste formaat hebben	De controleur die bij de deur checkt of een formulier correct is ingevuld
asyncpg	Library die supersnel met de PostgreSQL-database praat	De loopjongen tussen keuken en voorraadkast die meerdere bestellingen tegelijk kan halen
python-dotenv	Leest instellingen (zoals wachtwoorden) uit een apart configuratiebestand	De kluis met de sleutels: staan niet in de code zelf
OpenAI SDK	Library om met het AI-model te praten (voor de chat)	De telefoonlijn naar de AI-assistent

Wat dit samen doet: Uvicorn neemt een verzoek aan, FastAPI stuurt het naar het juiste loket, Pydantic controleert de invoer, asyncpg haalt de cijfers uit de database, en het antwoord gaat terug naar de browser. Voor de chat belt de OpenAI SDK het AI-model.

De opslag: de database

Bouwsteen	Wat het is	In gewone taal
PostgreSQL	Een database: een zeer betrouwbare digitale ladekast voor grote	De voorraadkast met genummerde laden, waar elke meting netjes op tijd en

	hoeveelheden cijfers	plaats ligt
--	----------------------	-------------

De fabrieksmachines schrijven hun metingen (productie-tellingen, alarmen, palletstatus) via een ander systeem (Node-RED) in deze database. Optimax leest die metingen alleen; het schrijft er nooit iets in. Dat is een bewuste veiligheidskeuze.

De verpakking en het draaien

Bouwsteen	Wat het is	In gewone taal
Docker	Verpakt een programma met al zijn benodigdheden in één "container"	Een verhuisdoos waar alles in zit: op elke computer pak je hem uit en het werkt
Image	Het dichtgeplakte recept van zo'n doos	De bouwtekening; van één image maak je zoveel draaiende dozen als je wilt
Registry	Een opslagplek voor die images	Het magazijn waar de dozen klaarstaan om opgehaald te worden
IXON SecureEdge	Het kastje in de fabriek waarop alles draait, met beveiligde toegang op afstand	De minicomputer ter plekke, met een afgesloten gang (VPN) ernaartoe

Waarom dit handig is: dezelfde doos die op de laptop van de bouwer werkt, draait identiek op de logger in de fabriek. Geen verrassingen door verschillen tussen computers.

3. Hoe een vraag door het systeem reist

Stel: je opent het dashboard en kijkt naar de productie van gisteren. Dit gebeurt er, stap voor stap:

1. Je opent <http://192.168.23.254:9000> in de browser (via de VPN).
2. De server stuurt eerst de webpagina zelf (de React-bestanden). De browser tekent de lay-out.
3. De browser vraagt vervolgens automatisch de cijfers op, bijvoorbeeld bij het loket </api/production/summary>.
4. De portier (Uvicorn) neemt het verzoek aan en checkt eerst de login (zie veiligheid hieronder).
5. Het juiste loket (FastAPI) draait een vaste databasevraag (SQL) via de loopjongen (asyncpg).
6. PostgreSQL geeft de cijfers terug; het loket rekent de KPI's uit (totaal, piekuur, stilstand, OEE).
7. Het antwoord gaat als nette gegevens terug naar de browser.
8. Recharts tekent er een grafiek van. Jij ziet het resultaat.

Dit hele rondje duurt normaal een fractie van een seconde. Gaat er onderweg iets mis (database even weg, vraag duurt te lang), dan toont de pagina een nette foutmelding met een knop om het opnieuw te proberen, in plaats van eindeloos te blijven laden.

4. Hoe de logging werkt

"Logging" is het dagboek van het systeem: bij elke gebeurtenis schrijft Optimax een regeltje weg. Waarom dat belangrijk is: als er ooit iets misgaat, kun je teruglezen wat er gebeurde, in plaats van te moeten gokken.

Hoe Optimax dit doet:

- **Elke regel is gestructureerd (JSON).** In plaats van losse zinnen schrijft Optimax nette regeltjes met vaste velden: tijdstip, ernst (info/waarschuwing/fout), en een omschrijving. Daardoor zijn ze later makkelijk te doorzoeken en te filteren.
- **Een rode draad per verzoek (request-id).** Elk binnenkomend verzoek krijgt een uniek volgnummer. Alle logregels van dat ene verzoek dragen dat nummer. Zo kun je één klacht ("om 10:42 ging het mis") als een aaneengesloten spoor terugvolgen.
- **Het dagboek loopt niet vol (rotatie).** De logs worden weggeschreven naar bestanden van maximaal 5 MB; er worden er 5 bewaard (samen max 25 MB). Is de jongste vol, dan begint een nieuwe en valt de oudste af. Zo kan het dagboek nooit de schijf van het kleine apparaat vol laten lopen.

Naast het dagboek heeft Optimax drie "meet-loketten" waar je de gezondheid kunt opvragen:

Loket	Wat het vertelt
<code>/api/health</code>	Leeft het dashboard, en is de database bereikbaar? (gebruikt door de automatische bewaking)
<code>/api/version</code>	Welke exacte versie draait er nu? (handig bij support)
<code>/api/metrics</code>	Hoeveel verzoeken, hoeveel fouten, hoe snel? (de tellerstanden)

De grotere monitoring-keten (staat klaar, nog niet aan)

Voor uitgebreidere bewaking is een set extra gereedschap voorbereid. De rolverdeling, met een analogie:

- **Promtail** = de koerier: leest de logbestanden en stuurt elke regel door.
- **Loki** = het archief: bewaart die regels doorzoekbaar.
- **Grafana** = de zoekbalk met dashboards: waar je de logs en grafieken bekijkt.

Belangrijk: deze keten staat klaar maar draait nu nog niet; hij is voorbereid voor later. Als hij aangaat, is er alvast een filter ingebouwd dat per ongeluk gelogde wachtwoorden of sleutels onleesbaar maakt (**REDACTED**) voordat ze worden opgeslagen.

Monitoren op afstand: de uitdaging

Omdat de logger achter een VPN zit, kan een gewone bewakingsdienst er van buitenaf niet bij. De oplossing is het omdraaien: het apparaat "belt" zelf met vaste tussenpozen naar buiten ("ik leef nog"). Stopt dat belletje, dan slaat er buiten een alarm af. Dat vangt zelfs een volledig uitgevallen apparaat. De uitgewerkte opties staan in een apart document ([optimax-monitoring-opties.md](#)).

5. De beveiligingslagen, in gewone taal

Optimax is doorgelicht op veiligheid. De belangrijkste maatregelen, zonder jargon:

- **Inloggen op het dashboard.** Wie de pagina opent, moet een gebruikersnaam en wachtwoord invoeren. Zonder dat start het dashboard zelfs niet (een bewuste veilige standaard).

- **Slot tegen wachtwoord-raden.** Probeert iemand veel wachtwoorden achter elkaar, dan gaat dat IP-adres tijdelijk op slot. De gezondheidscheck blijft wel altijd bereikbaar.
- **Poortwachter op de AI-chat.** De chat mag alleen lezen, nooit wijzigen. Een aparte controleur weigert elke databasevraag die niet puur opvragend is, begrenst het aantal rijen, en kapt vragen af die te lang duren of te vaak komen. Er is ook een dagbudget: is dat op, dan zegt de chat dat netjes en blijft het dashboard gewoon werken.
- **Geen wachtwoorden in het pakket.** De verhuisdoos (image) bevat zelf geen wachtwoorden. Die worden pas meegegeven op het moment dat de container wordt aangezet, en blijven zo in de afgeschermdede beheeromgeving.
- **Beschermkapjes op de webpagina (headers).** Standaard-instructies aan de browser die voorkomen dat de pagina in een ander, kwaadwillend kader wordt geladen of dat bestanden verkeerd worden uitgelegd.

6. Veelgestelde "waarom"-vragen

Waarom niet gewoon Grafana, dat draait toch al op de logger? Voor puur "cijfers op een scherm" had dat gekund. Optimax is bewust meer: een eigen, herkenbaar product met de DGS-huisstijl, een AI-chat op de eigen data, en een uniek fabrieksschema dat de doorstroom van de lijnen toont. Dat bouw je niet na in standaard Grafana-grafiekjes.

Waarom draait alles op dat ene kastje en niet in de cloud? De data staat in de fabriek en is gevoelig. Door alles lokaal te draaien blijft de data binnen, en werkt het dashboard ook als de internetverbinding wegvalt.

Wat gebeurt er als de database even uitvalt (stroomstoring)? Het dashboard valt niet om. Het probeert automatisch, met rustige tussenpozen, opnieuw verbinding te maken zodra de database weer opkomt. In de tussentijd toont het een nette melding.

7. Woordenlijst

Term	In gewone taal
API	Een verzameling "loketten" waar een programma vragen kan stellen aan een ander programma
Backend / frontend	Achterkant (de motor, onzichtbaar) / voorkant (wat je in de browser ziet)
Container / image	Een draaiende verhuisdoos met een programma erin / de dichtgeplakte bouwtekening daarvan
Database	Een digitale ladekast voor grote hoeveelheden gegevens
Endpoint / loket	Eén specifiek adres waar je één soort vraag kunt stellen
HTTP	De taal waarin browser en server met elkaar praten
JSON	Een net, gestructureerd formaat voor gegevens (vaste velden met waarden)
KPI	Kengetal, bijvoorbeeld totale productie, stilstand of OEE

Library	Kant-en-klare software die je hergebruikt in plaats van zelf te bouwen
Logging	Het dagboek van het systeem: wat gebeurde er, wanneer
OEE	Maat voor hoe effectief een machine draait (beschikbaarheid x prestatie x kwaliteit)
Poort (bv. 9000)	Een genummerde "deur" op een computer waarachter één programma luistert
Registry	Het magazijn waar verpakte programma's (images) klaarstaan
SQL	De taal waarin je vragen stelt aan een database
VPN	Een afgesloten, versleutelde gang naar het apparaat in de fabriek

Vragen of een onderdeel dat hier nog ontbreekt? Dan vullen we dit document aan; het is bedoeld om mee te groeien naarmate Optimax verder wordt opgeleverd.